

Report 6: Map

Le Nguyen Minh Quang (2540093)

October 2, 2026

1 What is Map

From the slides, Map is the simplest parallel pattern:

- We apply the same function f to every element of an array.
- Each output only depends on one input. We do not need the neighbours.
- There is a one to one relationship between input and output.
- Because of this, it is very easy to run in parallel and we can get almost linear speedup.

On the GPU, one thread handles one element:

```
def kernel(src, dst):
    tid = ...
    dst[tid] = f(src[tid])
```

In this labwork, one element is one pixel. I use 2D blocks like in labwork 4, so each thread finds its pixel with `tidx` and `tidy` and applies f to it.

I ran the code on Google Colab with a Tesla T4 GPU. My image is 1200x750 pixels.

2 6a: Binarization

First I convert the image to gray with my grayscale kernel from labwork 4. Then every gray pixel becomes black or white using a threshold. I used `threshold = 128`. If the pixel is smaller than 128 it becomes 0 (black), otherwise it becomes 255 (white). I used 255 instead of 1 so we can see the image.

```
if src[tidy, tidx] < threshold:
    dst[tidy, tidx] = 0
else:
    dst[tidy, tidx] = 255
```

3 6b: Brightness control

To make the image brighter I add a value to every pixel (for each colour R, G, B). A negative value makes it darker. I used `value = 80`. The result must stay between 0 and 255, so I cut it.

```
for ch in range(3):
    v = src[tidy, tidx, ch] + value
    if v > 255:
        v = 255
    if v < 0:
        v = 0
    dst[tidy, tidx, ch] = v
```

4 6c: Blending two images

Blending mixes two images of the same size with a weight c :

$$Q(x, y) = c \cdot \text{image1}(x, y) + (1 - c) \cdot \text{image2}(x, y)$$

For the second image I flipped my image left to right, so both images have the same size. I used $c = 0.5$, so both images count the same.

```
for ch in range(3):
    p1 = src1[tidy, tidy, ch]
    p2 = src2[tidy, tidy, ch]
    dst[tidy, tidy, ch] = np.uint8(c * p1 + (1 - c) * p2)
```

5 Results

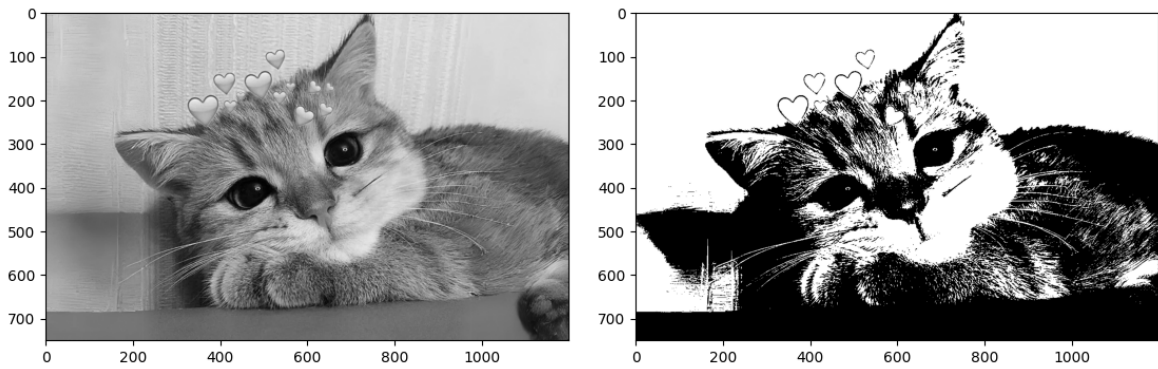


Figure 1: Gray image (left) and binarization with threshold 128 (right).

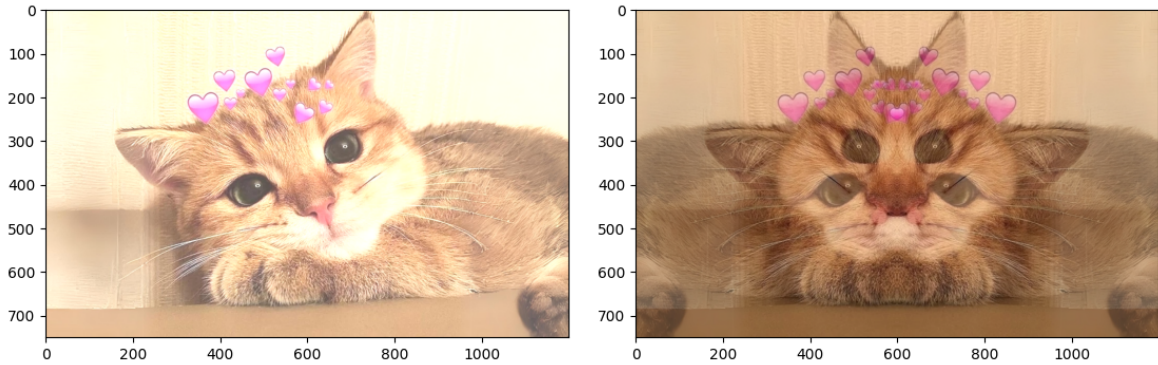


Figure 2: Brightness +80 (left) and blending with $c = 0.5$ (right).

6 Block size experiment

I tried 4 block sizes. One kernel call is very short, so I ran each kernel 50 times and took the average time.

Block	Binarization	Brightness	Blending
4x4	0.180 ms	0.162 ms	0.716 ms
8x8	0.044 ms	0.090 ms	0.362 ms
16x16	0.038 ms	0.065 ms	0.360 ms
32x32	0.042 ms	0.077 ms	0.397 ms

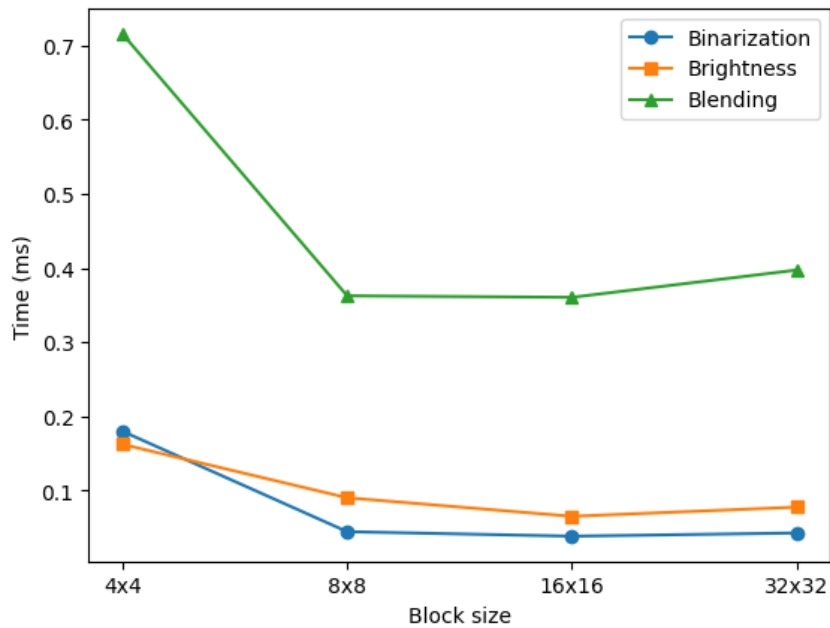


Figure 3: Block size vs time.

What I see:

- 4x4 is the slowest for all 3 kernels. A 4x4 block has only 16 threads, but a warp has 32 threads, so half of the warp is not used.
- 16x16 is the fastest for all 3 kernels. 8x8 and 32x32 are almost as fast.
- Binarization is the fastest because it reads only 1 value per pixel (gray). Brightness reads 3 values (R, G, B).
- Blending is the slowest. It reads two images, so it reads twice as much memory. It also uses decimal numbers ($c = 0.5$), and I think this is slower on the GPU than integer math.

7 Conclusion

I implemented 3 map kernels: binarization, brightness control and blending. They all follow the same idea from the slides: one thread takes one pixel and applies a function to it. The best block size for my image is 16x16, and blocks smaller than one warp (4x4) are slow.